

ALFIDO TECH INTERNSHIP PROGRAM

COHORT: MAY–JUNE 2026

- **Candidate Name:** Manish Kafle
- **Candidate ID:** BS/REG/120773
- **Domain:** Python Developer Intern
- **Project Task:** Task 4 – Full-Stack Student Registry Tracker (Flask)

1. Project Objective

This project is a fully responsive CRUD (Create, Read, Update, Delete) web application for managing student records. It uses Flask for the backend, MySQL with phpMyAdmin for data storage, and Bootstrap for a mobile-friendly user interface.

Key features include secure database operations, SQL-based analytics, dynamic search and filtering, and improved data handling through DictCursor, which enables reliable dictionary-based access to database records within templates.

2. Technical Stack Used

- Technical Stack
- **Backend Framework:** Flask 3.x
- **Database:** MySQL (managed with phpMyAdmin)
- **Database Connectivity:** Flask-MySQLdb
- **Environment Management:** Python-Dotenv
- **Template Engine:** Jinja2
- **Frontend/UI:** Bootstrap 5

3. Repository Links

- **Task Folder Repository Link:**
<https://github.com/CodedByManish/Alfido-python-internship/tree/main/task-04-student-registry>

4. Core Source Code (app.py)

```
import os
import MySQLdb.cursors
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_mysql import MySQL
from dotenv import load_dotenv

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv("FLASK_SECRET_KEY")

app.config['MYSQL_HOST'] = os.getenv("DB_HOST")
app.config['MYSQL_USER'] = os.getenv("DB_USER")
app.config['MYSQL_PASSWORD'] = os.getenv("DB_PASSWORD")
app.config['MYSQL_DB'] = os.getenv("DB_NAME")

mysql = MySQL(app)

@app.route('/')
def index():
    search_query = request.args.get('search', '').strip()

    cur = mysql.connection.cursor(MySQLdb.cursors.DictCursor)

    if search_query:
        query = "SELECT * FROM students WHERE name LIKE %s OR department LIKE %s ORDER BY id DESC"
        cur.execute(query, (f"%{search_query}%", f"%{search_query}%"))
    else:
        cur.execute("SELECT * FROM students ORDER BY id DESC")
    students = cur.fetchall()

    stats_cur = mysql.connection.cursor()
    stats_cur.execute("SELECT COUNT(*), AVG(marks), MAX(marks) FROM students")
    stats_raw = stats_cur.fetchone()

    stats = {
        "total": stats_raw[0] if stats_raw[0] else 0,
        "avg_marks": round(stats_raw[1], 1) if stats_raw[1] else 0,
        "top_score": stats_raw[2] if stats_raw[2] else 0
    }
    stats_cur.close()
    cur.close()
    return render_template('index.html', students=students, stats=stats, search_query=search_query)

@app.route('/add', methods=['GET', 'POST'])
def add_student():
    if request.method == 'POST':
        name = request.form['name'].strip()
        dept = request.form['department'].strip()
        marks = request.form['marks']
        attendance = request.form['attendance']

        if not name or not dept or not marks or not attendance:
            flash("All fields are strictly required.", "danger")
            return redirect(url_for('add_student'))

        cur = mysql.connection.cursor()
        cur.execute(
            "INSERT INTO students (name, department, marks, attendance) VALUES(%s, %s, %s, %s)",
            (name, dept, marks, attendance)
        )
        mysql.connection.commit()
        cur.close()
        flash(f"Record for {name} has been securely enrolled.", "success")
        return redirect(url_for('index'))

    return render_template('add_student.html')

@app.route('/delete/<int:id>')
def delete_student(id):
    cur = mysql.connection.cursor()
    cur.execute("DELETE FROM students WHERE id = %s", (id,))
    mysql.connection.commit()
    cur.close()
    flash("Student record successfully deleted from the registry.", "dark")
    return redirect(url_for('index'))

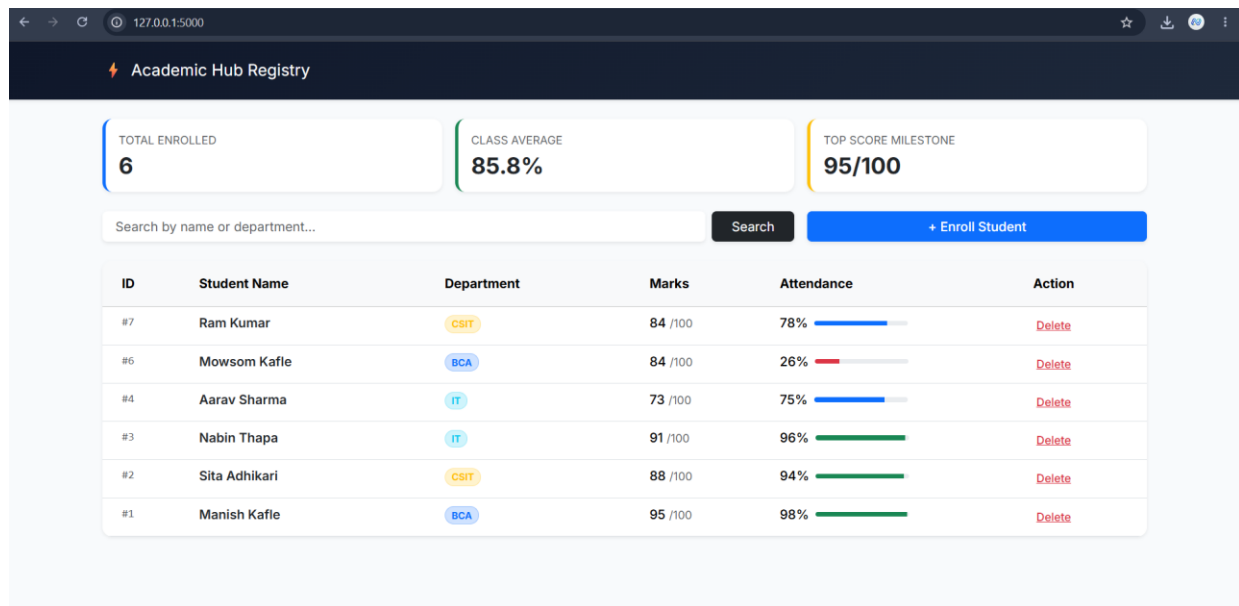
if __name__ == '__main__':
    app.run(debug=True)
```

5. Execution Screenshots & Explanations

5. Execution Screenshots & Explanations

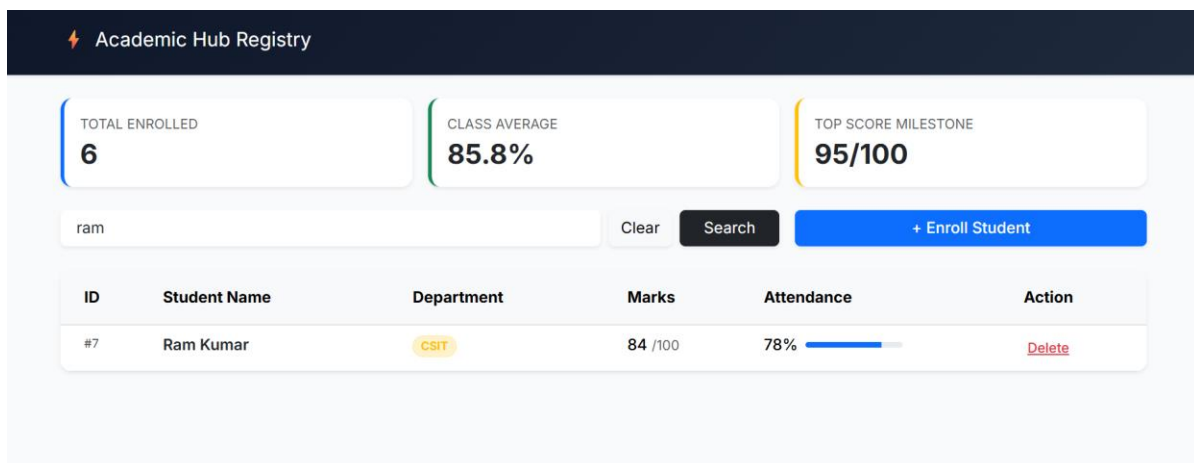
Case A: Dashboard Analytics

Description: The application running locally with a responsive dashboard displaying key metrics such as Total Students, Average Marks, and Top Score, calculated dynamically using SQL aggregate functions.



Case B: Dynamic Search Filter

Description: Demonstrates the search functionality, where keyword-based filtering retrieves matching student records using SQL LIKE queries and updates the displayed results in real time.



Case C: Secure Record Enrollment (Add Student Form)

Description: Verification of the creation layout engine. The interface provides a clean, validated web form matching the database schema. When submitted via a POST request, the backend performs data validation blocks, securely updates the MySQL table, and triggers a success message before redirecting back to the index view.

⚡ Academic Hub Registry

Enroll New Student

Input academic records into the database engine.

Full Name

newly added

Department Track

BCA

Marks obtained

50

Attendance Rate (%)

55

[Back to Dashboard](#)

Save to Matrix

Case D: Database Records in phpMyAdmin

Description: Shows the student records stored in the MySQL database, verifying successful data persistence and retrieval through the Flask-MySQLdb connection.

					id	name	department	marks	attendance
☐	Edit	Copy	Delete		1	Manish Kafle	BCA	95	98
☐	Edit	Copy	Delete		2	Sita Adhikari	CSIT	88	94
☐	Edit	Copy	Delete		3	Nabin Thapa	IT	91	96
☐	Edit	Copy	Delete		4	Aarav Sharma	IT	73	75
☐	Edit	Copy	Delete		6	Mowsom Kafle	BCA	84	26
☐	Edit	Copy	Delete		7	Ram Kumar	CSIT	84	78
☐	Edit	Copy	Delete		8	newly added	BCA	50	55
	☐ Check all	With selected:		Edit	Copy	Delete	Export		